SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 40%

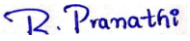
QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyse	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate
7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze

8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

---

Code of Conduct:

*I **R.Pranathi(192525076)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	All calculations accurate;	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
	correct final answer			

Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1. Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used.

   Bloom's Level: BL3 – Apply

ANS:

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database department;
Query OK, 1 row affected (0.01 sec)

mysql> use department;
Database changed
mysql> create table department(deptid int primary key,deptname varchar(30));
Query OK, 0 rows affected (0.12 sec)

mysql> create table student(studentid int primary key,name varchar(30),deptid int,marks int,foreign key(deptid) references department(deptid));
Query OK, 0 rows affected (0.18 sec)

mysql> insert into department values(10,'CSE'),(20,'ECE'),(30,'EEE');
Query OK, 3 rows affected (0.06 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> select*from department;
+--------+----------+
| deptid | deptname |
+--------+----------+
|     10 | CSE      |
|     20 | ECE      |
|     30 | EEE      |
+--------+----------+
3 rows in set (0.04 sec)

mysql> insert into student values(101,'sonu',10,85),(102,'pranathi',20,90),(103,'roopa',10,75),(104,'abhi',30,80);
Query OK, 4 rows affected (0.06 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> select*from student;
+-----------+----------+--------+-------+
| studentid | name     | deptid | marks |
+-----------+----------+--------+-------+
|       101 | sonu     |     10 |    85 |
|       102 | pranathi |     20 |    90 |
|       103 | roopa    |     10 |    75 |
|       104 | abhi     |     30 |    80 |
+-----------+----------+--------+-------+
4 rows in set (0.00 sec)
```

```
mysql> select*from student;
+-----------+----------+--------+-------+
| studentid | name     | deptid | marks |
+-----------+----------+--------+-------+
|       101 | sonu     |     10 |    85 |
|       102 | pranathi |     20 |    90 |
|       103 | roopa    |     10 |    75 |
|       104 | abhi     |     30 |    80 |
+-----------+----------+--------+-------+
4 rows in set (0.00 sec)

mysql> select student.name,department.deptname from student inner join department on student.deptid=department.deptid;
+----------+----------+
| name     | deptname |
+----------+----------+
| sonu     | CSE      |
| roopa    | CSE      |
| pranathi | ECE      |
| abhi     | EEE      |
+----------+----------+
4 rows in set (0.00 sec)

mysql> select deptid from department where deptname='CSE';
+--------+
| deptid |
+--------+
|     10 |
+--------+
1 row in set (0.05 sec)

mysql> select name from student where deptid=10;
+-------+
| name  |
+-------+
| sonu  |
| roopa |
+-------+
2 rows in set (0.00 sec)
```

Logic used:

- The JOIN query is used to combine data from two related tables using the common attribute DeptID and display information from both tables in a single result.
- The SUBQUERY first retrieves the required DeptID from the Department table, and then the outer query uses that result to fetch the corresponding student records.
- Both approaches retrieve related information, but JOIN is generally preferred for combining data from multiple tables, while SUBQUERY is useful when the result of one query is required as input for another query.

2. **Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations.**

   Bloom's Level: BL4 – Analyse

ANS:

Example Database Schema

STUDENT Table

StudentID	Name	DeptID	Marks
101	Sonu	10	85
102	Pranathi	20	90
103	Roops	10	78
104	Abhighna	30	88

DEPARTMENT Table

DeptID	DeptName
10	CSE
20	ECE
30	EEE

Relational Algebra Operators

Operator	Meaning
σ (Selection)	Selects rows satisfying a condition
π (Projection)	Selects required columns
⋈ (Join)	Combines two relations
∪ (Union)	Combines tuples from two relations
− (Difference)	Returns tuples present in one relation but not another
× (Cartesian Product)	Combines every row with every other row

Query 1

Retrieve the names of students who belong to the CSE department.

Step 1: Join Student and Department

Student ⋈ Student.DeptID = Department.DeptID Department

Step 2: Select only CSE department

σ DeptName='CSE'

(Student ⋈ Department)

Step 3: Display only student names

π Name

(

σ DeptName='CSE'

(Student ⋈ Department)

)

Explanation

- Join both relations using DeptID.
- Select records where Department is CSE.
- Project only the Name attribute.

Query 2

Retrieve students who scored more than 80 marks.

Relational Algebra

π Name

(

σ Marks>80(Student)

)

Explanation

- Selection chooses students with Marks greater than 80.
- Projection displays only their names.

Query 3

Retrieve CSE students who scored above 80 marks.

Relational Algebra

π Name

(

σ DeptName='CSE' ∧ Marks>80

(Student ⋈ Department)

)

Explanation

This query uses two conditions:

- Department must be CSE.
- Marks must be greater than 80.

After joining the relations, both conditions are applied together**.**

Query 4

Retrieve Department Name and Student Name.

Relational Algebra

π Name, DeptName

(

Student ⋈ Student.DeptID = Department.DeptID Department

)

Explanation

- Join Student and Department tables.
- Display only Name and Department Name.

Query 5

Retrieve students belonging to ECE department whose marks are greater than 85.

Relational Algebra

π Name

(

σ DeptName='ECE' ∧ Marks>85

(Student ⋈ Department)

)

Explanation

- Join the relations.
- Select students satisfying both conditions.
- Display only their names.

Query 6

Retrieve all departments having students with marks above 80.

Relational Algebra

π DeptName

(

σ Marks>80

(Student $\bowtie$ Department)

)

Explanation

- Join Student and Department tables.
- Select students with Marks greater than 80.
- Display only department names.

Query 7

Retrieve all student names and marks from the CSE and ECE departments.

Relational Algebra

π Name, Marks

(

σ DeptName='CSE' $\vee$ DeptName='ECE'

(Student $\bowtie$ Department)

)

Explanation

- Join the relations.
- Select records where department is either CSE or ECE.
- Project Name and Marks.

3. **Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates.**

   Bloom's Level: BL6 – Create

ANS:

A Student Management System stores information about students, departments, and courses. Each student belongs to one department and can enroll in multiple courses.

```
mysql> create database department;
Query OK, 1 row affected (0.09 sec)

mysql> use department;
Database changed
mysql> CREATE TABLE Department(DeptID INT PRIMARY KEY, DeptName VARCHAR(30));
Query OK, 0 rows affected (0.08 sec)

mysql> CREATE TABLE Course(CourseID INT PRIMARY KEY, CourseName VARCHAR(40));
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE Student(StudentID INT PRIMARY KEY, StudentName VARCHAR(30), DeptID INT, Age INT, Marks INT, FOREIGN KEY (DeptID) REFERENCES Department(D
eptID));
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO Department VALUES (10,'CSE'), (20,'ECE'), (30,'EEE');
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Student VALUES (101,'tabassum',10,20,85), (102,'sonu',20,19,90), (103,'pranathi',10,21,78), (104,'roops',30,20,88);
Query OK, 4 rows affected (0.06 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> select*from student;
+-----------+-------------+--------+------+-------+
| StudentID | StudentName | DeptID | Age  | Marks |
+-----------+-------------+--------+------+-------+
|       101 | tabassum    |     10 |   20 |    85 |
|       102 | sonu        |     20 |   19 |    90 |
|       103 | pranathi    |     10 |   21 |    78 |
|       104 | roops       |     30 |   20 |    88 |
+-----------+-------------+--------+------+-------+
4 rows in set (0.00 sec)

mysql> SELECT Student.StudentName, Department.DeptName FROM Student INNER JOIN Department ON Student.DeptID = Department.DeptID;
+-------------+----------+
| StudentName | DeptName |
+-------------+----------+
| tabassum    | CSE      |
| pranathi    | CSE      |
```

```
mysql> SELECT Student.StudentName, Department.DeptName FROM Student INNER JOIN Department ON Student.DeptID = Department.DeptID;
+-------------+----------+
| StudentName | DeptName |
+-------------+----------+
| tabassum    | CSE      |
| pranathi    | CSE      |
| sonu        | ECE      |
| roops       | EEE      |
+-------------+----------+
4 rows in set (0.00 sec)

mysql> SELECT StudentName,Marks FROM Student WHERE Marks>80;
+-------------+-------+
| StudentName | Marks |
+-------------+-------+
| tabassum    |    85 |
| sonu        |    90 |
| roops       |    88 |
+-------------+-------+
3 rows in set (0.00 sec)

mysql> SELECT StudentName FROM Student WHERE DeptID= ( SELECT DeptID FROM Department WHERE DeptName='CSE' );
+-------------+
| StudentName |
+-------------+
| tabassum    |
| pranathi    |
+-------------+
2 rows in set (0.05 sec)

mysql> UPDATE Student SET Marks=90 WHERE StudentID=101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM Student WHERE StudentID=101;
+-----------+-------------+--------+------+-------+
| StudentID | StudentName | DeptID | Age  | Marks |
+-----------+-------------+--------+------+-------+
|       101 | tabassum    |     10 |   20 |    90 |
+-----------+-------------+--------+------+-------+
1 row in set (0.00 sec)

mysql> DELETE FROM Student WHERE Marks<80;
Query OK, 1 row affected (0.05 sec)
```

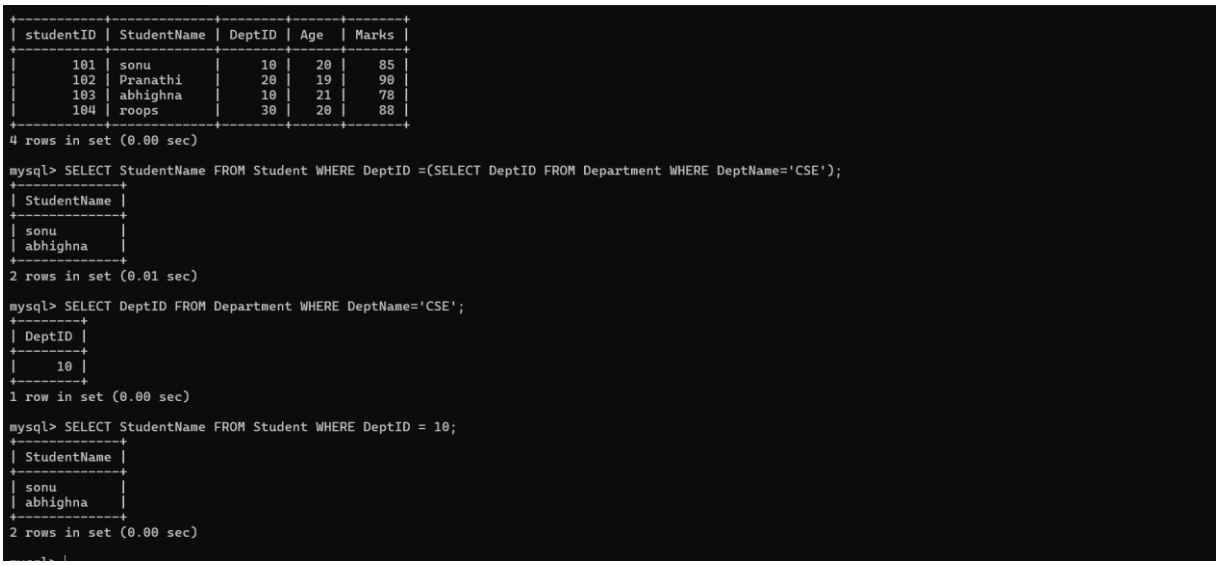
```
mysql> DELETE FROM Student WHERE Marks<80;
Query OK, 1 row affected (0.05 sec)

mysql> SELECT COUNT(*) FROM Student;
+----------+
| COUNT(*) |
+----------+
|        3 |
+----------+
1 row in set (0.01 sec)

mysql> SELECT AVG(Marks) FROM Student;
+------------+
| AVG(Marks) |
+------------+
|    89.3333 |
+------------+
1 row in set (0.00 sec)

mysql> SELECT MAX(Marks) FROM Student;
+------------+
| MAX(Marks) |
+------------+
|         90 |
+------------+
1 row in set (0.05 sec)

mysql> SELECT MIN(Marks) FROM Student;
+------------+
| MIN(Marks) |
+------------+
|         88 |
+------------+
1 row in set (0.00 sec)

mysql> SELECT StudentName,Marks FROM Student ORDER BY Marks DESC;
+-------------+-------+
| StudentName | Marks |
+-------------+-------+
| tabassum    |    90 |
| sonu        |    90 |
| roops       |    88 |
+-------------+-------+
3 rows in set (0.00 sec)
```

4. **Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution.**

   Bloom's Level: BL4 – Analyze

ANS:

```
mysql> create database department;
Query OK, 1 row affected (0.07 sec)

mysql> use department;
Database changed
mysql> CREATE TABLE Department(DeptID INT PRIMARY KEY,DeptName VARCHAR(30));
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE TABLE student(studentID INT PRIMARY KEY,StudentName VARCHAR(30),DeptID INT,Age INT,Marks INT,FOREIGN KEY(Dept ID)REFERENCES Department(DeptID)
);
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'I
D)REFERENCES Department(DeptID))' at line 1
mysql> CREATE TABLE student(studentID INT PRIMARY KEY,StudentName VARCHAR(30),DeptID INT,Age INT,Marks INT,FOREIGN KEY(DeptID)REFERENCES Department(DeptID))
;
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO Department VALUES (10,'CSE'), (20,'ECE'), (30,'EEE');
Query OK, 3 rows affected (0.05 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Department;
+--------+----------+
| DeptID | DeptName |
+--------+----------+
|     10 | CSE      |
|     20 | ECE      |
|     30 | EEE      |
+--------+----------+
3 rows in set (0.01 sec)

mysql> INSERT INTO Student VALUES(101,'sonu',10,20,85),(102,'Pranathi',20,19,90),(103,'abhighna',10,21,78),(104,'roops',30,20,88);
Query OK, 4 rows affected (0.02 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Student;
+-----------+-------------+--------+------+-------+
| studentID | StudentName | DeptID | Age  | Marks |
+-----------+-------------+--------+------+-------+
|       101 | sonu        |     10 |   20 |    85 |
|       102 | Pranathi    |     20 |   19 |    90 |
|       103 | abhighna    |     10 |   21 |    78 |
|       104 | roops       |     30 |   20 |    88 |
+-----------+-------------+--------+------+-------+
4 rows in set (0.00 sec)
```



```
+-----------+-------------+--------+------+-------+
| studentID | StudentName | DeptID | Age  | Marks |
+-----------+-------------+--------+------+-------+
|       101 | sonu        |     10 |   20 |    85 |
|       102 | Pranathi    |     20 |   19 |    90 |
|       103 | abhighna    |     10 |   21 |    78 |
|       104 | roops       |     30 |   20 |    88 |
+-----------+-------------+--------+------+-------+
4 rows in set (0.00 sec)

mysql> SELECT StudentName FROM Student WHERE DeptID =(SELECT DeptID FROM Department WHERE DeptName='CSE');
+-------------+
| StudentName |
+-------------+
| sonu        |
| abhighna    |
+-------------+
2 rows in set (0.01 sec)

mysql> SELECT DeptID FROM Department WHERE DeptName='CSE';
+--------+
| DeptID |
+--------+
|     10 |
+--------+
1 row in set (0.00 sec)

mysql> SELECT StudentName FROM Student WHERE DeptID = 10;
+-------------+
| StudentName |
+-------------+
| sonu        |
| abhighna    |
+-------------+
2 rows in set (0.00 sec)
```

Step-by-Step Execution

Step 1: SQL Reads the Entire Query

The DBMS first reads the complete SQL statement and identifies that there is a nested subquery inside the WHERE clause.

Explanation:

- The query contains an outer query and an inner query.
- SQL always executes the inner query first because the outer query depends on its result.

Step 2: Execute the Inner Query

SQL executes the inner query.

SELECT DeptID

FROM Department

WHERE DeptName='CSE';

Department Table

DeptID	DeptName
10	CSE
20	ECE
30	EEE

Output

DeptID
10

Explanation:

- SQL searches the Department table.
- It checks each row to find DeptName = 'CSE'.
- It finds the matching row and returns DeptID = 10.
- This value is temporarily stored in memory.

Step 3: Pass the Result to the Outer Query

The value returned by the inner query (10) is passed to the outer query.

The original query

```
SELECT StudentName
FROM Student
WHERE DeptID =
(
SELECT DeptID
FROM Department
WHERE DeptName='CSE'
);
```

becomes

```
SELECT StudentName
FROM Student
WHERE DeptID = 10;
```

Explanation:

- SQL replaces the entire subquery with the value returned by the inner query.
- Now it becomes a normal SELECT statement.

Step 4: Execute the Outer Query

SQL now searches the Student table.

Student Table

StudentID	StudentName	DeptID	Marks
101	Sonu	10	85
102	Pranathi	20	90
103	abhighna	10	78
104	roops	30	88

SQL checks each record one by one.

- Sonu → DeptID = 10 ✓ Selected
- Pranathi → DeptID = 20 ✗ Not Selected
- Abhighna → DeptID = 10 ✓ Selected
- roops → DeptID = 30 ✗ Not Selected

Step 5: Retrieve the Matching Records

Only the records satisfying the condition DeptID = 10 are retrieved.

StudentName
Sonu
abhighna

Explanation:

- Only students belonging to the CSE department are selected.
- Records with other department IDs are ignored.

Step 6: Display the Final Output

Final Output

StudentName
Sonu
abhighna

Explanation:

- SQL displays the result of the outer query.
- The execution of the nested query is complete.

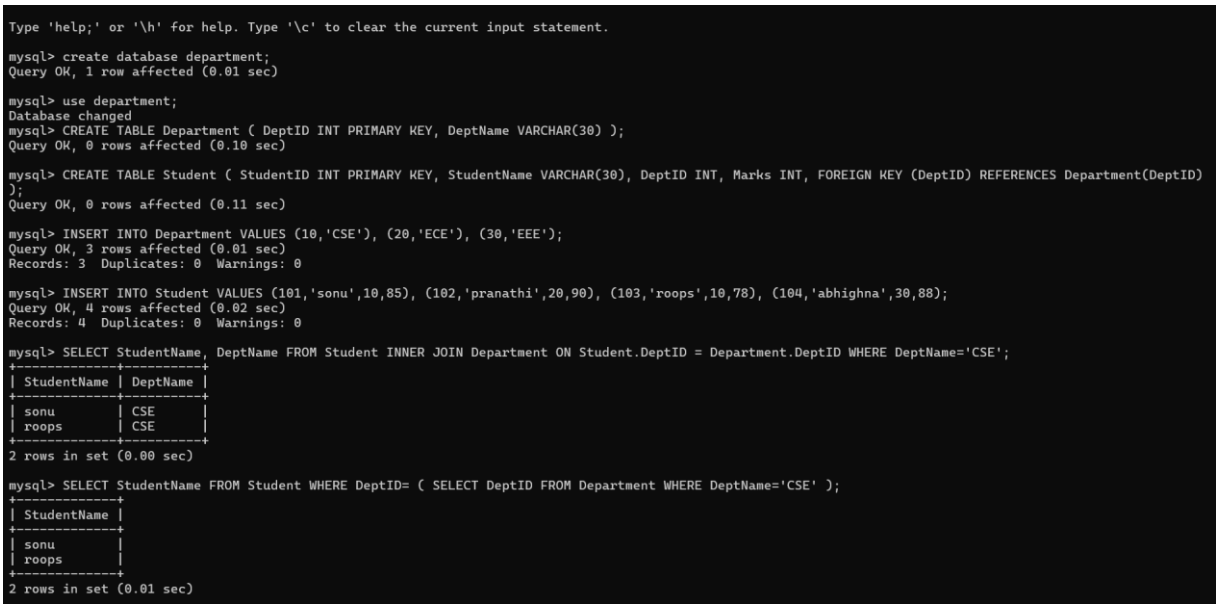
5. **Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate**

ANS:

SQL provides different methods to retrieve the same information from a database. Two common approaches are **JOIN** and **SUBQUERY**.

- **JOIN** combines data from two or more related tables using a common column.
- **SUBQUERY** is a query written inside another query, where the result of the inner query is used by the outer query.

Both approaches can produce the same output, but their performance and usage differ.



```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database department;
Query OK, 1 row affected (0.01 sec)

mysql> use department;
Database changed
mysql> CREATE TABLE Department ( DeptID INT PRIMARY KEY, DeptName VARCHAR(30) );
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE TABLE Student ( StudentID INT PRIMARY KEY, StudentName VARCHAR(30), DeptID INT, Marks INT, FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
Query OK, 0 rows affected (0.11 sec)

mysql> INSERT INTO Department VALUES (10,'CSE'), (20,'ECE'), (30,'EEE');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Student VALUES (101,'sonu',10,85), (102,'pranathi',20,90), (103,'roops',10,78), (104,'abhighna',30,88);
Query OK, 4 rows affected (0.02 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT StudentName, DeptName FROM Student INNER JOIN Department ON Student.DeptID = Department.DeptID WHERE DeptName='CSE';
+-------------+----------+
| StudentName | DeptName |
+-------------+----------+
| sonu        | CSE      |
| roops       | CSE      |
+-------------+----------+
2 rows in set (0.00 sec)

mysql> SELECT StudentName FROM Student WHERE DeptID= ( SELECT DeptID FROM Department WHERE DeptName='CSE' );
+-------------+
| StudentName |
+-------------+
| sonu        |
| roops       |
+-------------+
2 rows in set (0.01 sec)
```

Feature	JOIN	SUBQUERY
Definition	Combines data from two or more related tables.	A query written inside another query.
Execution	Single query execution.	Inner query executes first, then outer query.
Performance	Generally faster.	Generally slower.
Memory Usage	Lower.	May require more memory.
Readability	Easy for multiple tables.	Easy for simple conditions.
Large Databases	Highly efficient.	Can become slow for large datasets.
Optimization	Well optimized by DBMS.	Less optimized in some cases.
Best Use	Combining related tables.	When one query depends on another.

Efficiency Evaluation

JOIN

Advantages

- Faster execution.
- Retrieves data from multiple tables in one operation.
- Better optimized by the DBMS.
- Suitable for large databases.
- Reduces query execution time.

Disadvantages

- Slightly difficult for beginners.
- Complex JOINs can become lengthy.

SUBQUERY

Advantages

- Easy to understand.
- Useful when one query depends on another.
- Suitable for simple filtering operations.
- Improves logical readability.

Disadvantages

- Slower than JOIN for large datasets.
- Nested execution increases processing time.
- Multiple subqueries reduce performance.

Criteria	JOIN	SUBQUERY
Execution Time	Fast	Slower
Query Optimization	Excellent	Good
Scalability	Excellent	Moderate
Suitable for Large Data	Yes	Limited
Suitable for Complex Conditions	Yes	Yes
Preferred by DBMS	Yes	Less Preferred

Which Approach is Better?

For this problem,

JOIN is the better approach because:

- It combines related tables in one operation.
- The database optimizer executes JOIN efficiently.
- It performs better when the database contains thousands or millions of records.
- It reduces execution time and improves performance.

Subqueries are useful when the result of one query is required by another query, but they are generally slower for large datasets.

6. **Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction.**

   Bloom's Level: BL6 – Create

ANS:

A View is a virtual table created from one or more database tables. It does not store data physically; instead, it stores the SQL query used to retrieve the data. Views are mainly used to restrict access to sensitive information, simplify complex queries, and provide data abstraction.

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database department;
Query OK, 1 row affected (0.11 sec)

mysql> use department;
Database changed
mysql> CREATE TABLE Department ( DeptID INT PRIMARY KEY, DeptName VARCHAR(30) );
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE TABLE Student ( StudentID INT PRIMARY KEY, StudentName VARCHAR(30), DeptID INT, Marks INT, FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO Department VALUES (10,'CSE'), (20,'ECE'), (30,'EEE');
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Student VALUES(101,'sonu',10,85),(102,'pranathi',20,90),(103,'roops',10,78),(104,'abhighna',30,88);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Student;
+-----------+-------------+--------+-------+
| StudentID | StudentName | DeptID | Marks |
+-----------+-------------+--------+-------+
|       101 | sonu        |     10 |    85 |
|       102 | pranathi    |     20 |    90 |
|       103 | roops       |     10 |    78 |
|       104 | abhighna    |     30 |    88 |
+-----------+-------------+--------+-------+
4 rows in set (0.00 sec)

mysql> CREATE VIEW Student_View AS SELECT StudentID, StudentName, DeptID FROM Student;
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> SELECT * FROM Student;
+-----------+-------------+--------+-------+
| StudentID | StudentName | DeptID | Marks |
+-----------+-------------+--------+-------+
|       101 | sonu        |     10 |    85 |
|       102 | pranathi    |     20 |    90 |
|       103 | roops       |     10 |    78 |
|       104 | abhighna    |     30 |    88 |
+-----------+-------------+--------+-------+
4 rows in set (0.00 sec)

mysql> CREATE VIEW Student_View AS SELECT StudentID, StudentName, DeptID FROM Student;
Query OK, 0 rows affected (0.04 sec)

mysql> SELECT * FROM Student_View;
+-----------+-------------+--------+
| StudentID | StudentName | DeptID |
+-----------+-------------+--------+
|       101 | sonu        |     10 |
|       102 | pranathi    |     20 |
|       103 | roops       |     10 |
|       104 | abhighna    |     30 |
+-----------+-------------+--------+
4 rows in set (0.01 sec)

mysql> CREATE VIEW Student_Department_View AS SELECT Student.StudentName, Department.DeptName FROM Student INNER JOIN Department ON Student.DeptID=Departmen
t.DeptID;
Query OK, 0 rows affected (0.14 sec)

mysql> SELECT * FROM Student_Department_View;
+-------------+----------+
| StudentName | DeptName |
+-------------+----------+
| sonu        | CSE      |
| roops       | CSE      |
| pranathi    | ECE      |
| abhighna    | EEE      |
+-------------+----------+
4 rows in set (0.00 sec)
```

```
mysql>  UPDATE Student_View SET StudentName='venkat rao' WHERE StudentID=101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM Student_View;
+-----------+-------------+--------+
| StudentID | StudentName | DeptID |
+-----------+-------------+--------+
|       101 | venkat rao  |     10 |
|       102 | pranathi    |     20 |
|       103 | roops       |     10 |
|       104 | abhighna    |     30 |
+-----------+-------------+--------+
4 rows in set (0.00 sec)

mysql> DROP VIEW Student_View;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE VIEW Student_View AS SELECT StudentID, StudentName, DeptID FROM Student;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM Student_View;
+-----------+-------------+--------+
| StudentID | StudentName | DeptID |
+-----------+-------------+--------+
|       101 | venkat rao  |     10 |
|       102 | pranathi    |     20 |
|       103 | roops       |     10 |
|       104 | abhighna    |     30 |
+-----------+-------------+--------+
4 rows in set (0.00 sec)
```

Justification for Security

Views improve database security because:

- Hide confidential columns such as salary, marks, passwords, and account balance.
- Allow users to access only authorized information.
- Prevent unauthorized viewing of sensitive data.
- Reduce the risk of accidental data exposure.
- Support role-based access control.

Justification for Data Abstraction

Views provide abstraction because:

- Hide the complexity of SQL queries.
- Users interact with a simple virtual table.
- No need to know the underlying table structure.
- Multiple tables can be represented as one view.
- Changes to the base tables usually do not affect users if the view remains unchanged.

Advantages of Views

- Provides data security.
- Restricts unauthorized access.
- Simplifies complex SQL queries.
- Improves data abstraction.
- Reduces query repetition.
- Enhances code reusability.
- Makes database management easier.
- Supports customized data presentation.

Disadvantages of Views

- Does not store data physically.
- Complex views may reduce performance.
- Some views cannot be updated.
- Depends on the underlying tables.

7. Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation.

Bloom's Level: BL4 – Analyze

ANS:

Relational Algebra is a procedural query language used to describe how data should be retrieved from a relational database. SQL (Structured Query Language) is a declarative language that specifies what data should be retrieved rather than how to retrieve it.

```
mysql> create database department;
Query OK, 1 row affected (0.01 sec)

mysql> use department;
Database changed
mysql> CREATE TABLE Department ( DeptID INT PRIMARY KEY, DeptName VARCHAR(30) );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Student ( StudentID INT PRIMARY KEY, StudentName VARCHAR(30), DeptID INT, Marks INT, FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO Department VALUES (10,'CSE'), (20,'ECE'), (30,'EEE');
Query OK, 3 rows affected (0.03 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Department;
+--------+----------+
| DeptID | DeptName |
+--------+----------+
|     10 | CSE      |
|     20 | ECE      |
|     30 | EEE      |
+--------+----------+
3 rows in set (0.00 sec)

mysql> INSERT INTO Student VALUES (101,'sonu',10,85), (102,'roops',20,90), (103,'pranathi',10,78), (104,'abhighna',30,88);
Query OK, 4 rows affected (0.05 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Student;
+-----------+-------------+--------+-------+
| StudentID | StudentName | DeptID | Marks |
+-----------+-------------+--------+-------+
|       101 | sonu        |     10 |    85 |
|       102 | roops       |     20 |    90 |
|       103 | pranathi    |     10 |    78 |
|       104 | abhighna    |     30 |    88 |
+-----------+-------------+--------+-------+
4 rows in set (0.00 sec)
```

```
|       102 | roops       |     20 |    90 |
|       103 | pranathi    |     10 |    78 |
|       104 | abhighna    |     30 |    88 |
+-----------+-------------+--------+-------+
4 rows in set (0.00 sec)

mysql> SELECT StudentName FROM Student INNER JOIN Department ON Student.DeptID=Department.DeptID WHERE DeptName='CSE';
+-------------+
| StudentName |
+-------------+
| sonu        |
| pranathi    |
+-------------+
2 rows in set (0.00 sec)

mysql> SELECT StudentName,Marks FROM Student WHERE Marks>80;
+-------------+-------+
| StudentName | Marks |
+-------------+-------+
| sonu        |    85 |
| roops       |    90 |
| abhighna    |    88 |
+-------------+-------+
3 rows in set (0.00 sec)

mysql> SELECT StudentName,DeptName FROM Student INNER JOIN Department ON Student.DeptID=Department.DeptID;
+-------------+-----------+
| StudentName | DeptName  |
+-------------+-----------+
| sonu        | CSE       |
| pranathi    | CSE       |
| roops       | ECE       |
| abhighna    | EEE       |
+-------------+-----------+
4 rows in set (0.00 sec)
```

Write the Relational Algebra Expression

Problem

Retrieve the names of students belonging to the CSE department.

Relational Algebra

π StudentName

(

σ DeptName='CSE'

(Student ⋈ Student.DeptID = Department.DeptID Department)

)

Explanation

- ⋈ (Join) combines Student and Department tables using DeptID.
- σ (Selection) filters only the records where Department Name is CSE.
- π (Projection) displays only the StudentName column.

Step 6: Convert into Equivalent SQL Query

SELECT StudentName

FROM Student

INNER JOIN Department

ON Student.DeptID = Department.DeptID

WHERE DeptName='CSE';

Output

StudentName
sonu
pranathi

Step 7: Second Example

Problem

Retrieve students who scored more than 80 marks.

Relational Algebra

π StudentName, Marks

(

σ Marks > 80 (Student)

)

Equivalent SQL Query

```
SELECT StudentName, Marks
FROM Student
WHERE Marks > 80;
```

Output

StudentName	Marks
sonu	85
pranathi	90
abhighna	88

Step 8: Third Example

Problem

Retrieve Student Name and Department Name.

Relational Algebra

π StudentName, DeptName

(

Student ⋈ Student.DeptID = Department.DeptID Department

)

Equivalent SQL Query

```
SELECT StudentName, DeptName
FROM Student
INNER JOIN Department
ON Student.DeptID = Department.DeptID;
```

Output

StudentName	DeptName
sonu	CSE
roops	ECE
pranathi	CSE
abhighna	EEE

Analysis of Differences in Representation

Relational Algebra	**SQL**
Uses mathematical symbols such as σ, π, and ⋈.	Uses SQL keywords such as SELECT, FROM, WHERE, and JOIN.
Procedural language that specifies how data is retrieved.	Declarative language that specifies what data is required.
Mainly used for theoretical query processing.	Used for practical database implementation.
Requires knowledge of algebraic operators.	Easy to understand using English-like syntax.
Forms the basis for query optimization.	Used to execute queries in DBMS software.

8. **Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements.**

Bloom's Level: BL5 – Evaluate

ANS:

SQL query optimization is the process of improving the performance of SQL queries by reducing execution time and resource usage. Optimization can be achieved using **JOINs**, **Indexes**, and **Views**. These techniques help the DBMS retrieve data more efficiently, especially for large databases.

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database department;
Query OK, 1 row affected (0.06 sec)

mysql> use department;
Database changed
mysql> CREATE TABLE Department ( DeptID INT PRIMARY KEY, DeptName VARCHAR(30) );
Query OK, 0 rows affected (0.08 sec)

mysql> CREATE TABLE Student ( StudentID INT PRIMARY KEY, StudentName VARCHAR(30), DeptID INT, Marks INT, FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
Query OK, 0 rows affected (0.12 sec)

mysql> INSERT INTO Department VALUES (10,'CSE'), (20,'ECE'), (30,'EEE');
Query OK, 3 rows affected (0.12 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Student VALUES (101,'Rahul',10,85), (102,'Priya',20,90), (103,'Arun',10,78), (104,'Sneha',30,88);
Query OK, 4 rows affected (0.05 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT StudentName FROM Student WHERE DeptID IN (SELECT DeptID FROM Department WHERE DeptName='CSE');
+-------------+
| StudentName |
+-------------+
| Rahul       |
| Arun        |
+-------------+
2 rows in set (0.05 sec)

mysql> SELECT StudentName, DeptName FROM Student INNER JOIN Department ON Student.DeptID=Department.DeptID WHERE DeptName='CSE';
+-------------+----------+
| StudentName | DeptName |
+-------------+----------+
| Rahul       | CSE      |
| Arun        | CSE      |
+-------------+----------+
2 rows in set (0.00 sec)
```

```
mysql> SELECT StudentName, DeptName FROM Student INNER JOIN Department ON Student.DeptID=Department.DeptID WHERE DeptName='CSE';
+-------------+----------+
| StudentName | DeptName |
+-------------+----------+
| Rahul       | CSE      |
| Arun        | CSE      |
+-------------+----------+
2 rows in set (0.00 sec)

mysql> CREATE INDEX idx_DeptID ON Student(DeptID);
Query OK, 0 rows affected (0.10 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE VIEW Student_Department_View AS SELECT Student.StudentName, Department.DeptName, Student.Marks FROM Student INNER JOIN Department ON Student.D
eptID=Department.DeptID;
Query OK, 0 rows affected (0.06 sec)

mysql> SELECT * FROM Student_Department_View;
+-------------+----------+-------+
| StudentName | DeptName | Marks |
+-------------+----------+-------+
| Rahul       | CSE      |    85 |
| Arun        | CSE      |    78 |
| Priya       | ECE      |    90 |
| Sneha       | EEE      |    88 |
+-------------+----------+-------+
4 rows in set (0.00 sec)
```

Performance Evaluation

Technique	Performance Improvement
Subquery	Slower execution because of nested queries.
JOIN	Faster execution by combining tables directly.
Index	Reduces search time by avoiding full table scans.
View	Simplifies repeated queries and improves maintainability.

9. Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem.

Bloom's Level: BL6 – Create

ANS:

In a Student Management System, the administrator may need to analyze students' performance department-wise. SQL provides GROUP BY, Aggregate Functions, and Nested Queries to generate reports and solve such real-world problems.

Aggregate functions such as COUNT(), SUM(), AVG(), MAX(), and MIN() summarize data, while nested queries help retrieve records based on computed values.

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database department;
Query OK, 1 row affected (0.09 sec)

mysql> use department;
Database changed
mysql> CREATE TABLE Department ( DeptID INT PRIMARY KEY, DeptName VARCHAR(30) );
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE Student ( StudentID INT PRIMARY KEY, StudentName VARCHAR(30), DeptID INT, Marks INT, FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
Query OK, 0 rows affected (0.11 sec)

mysql> INSERT INTO Department VALUES (10,'CSE'), (20,'ECE'), (30,'EEE');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Student VALUES (101,'Rahul',10,85), (102,'Priya',20,90), (103,'Arun',10,78), (104,'Sneha',30,88), (105,'Kiran',20,95), (106,'Anitha',10,8
2);
Query OK, 6 rows affected (0.06 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT Department.DeptName, AVG(Student.Marks) AS AverageMarks FROM Student INNER JOIN Department ON Student.DeptID = Department.DeptID GROUP BY Depa
rtment.DeptName;
+----------+--------------+
| DeptName | AverageMarks |
+----------+--------------+
| CSE      |      81.6667 |
| ECE      |      92.5000 |
| EEE      |      88.0000 |
+----------+--------------+
3 rows in set (0.05 sec)

mysql> SELECT Department.DeptName, COUNT(Student.StudentID) AS TotalStudents FROM Student INNER JOIN Department ON Student.DeptID = Department.DeptID GROUP
BY Department.DeptName;
+----------+---------------+
| DeptName | TotalStudents |
+----------+---------------+
| CSE      |             3 |
| ECE      |             2 |
| EEE      |             1 |
```

```
3 rows in set (0.05 sec)

mysql> SELECT Department.DeptName, COUNT(Student.StudentID) AS TotalStudents FROM Student INNER JOIN Department ON Student.DeptID = Department.DeptID GROUP
BY Department.DeptName;
+----------+---------------+
| DeptName | TotalStudents |
+----------+---------------+
| CSE      |             3 |
| ECE      |             2 |
| EEE      |             1 |
+----------+---------------+
3 rows in set (0.00 sec)

mysql> SELECT Department.DeptName, MAX(Student.Marks) AS HighestMarks FROM Student INNER JOIN Department ON Student.DeptID = Department.DeptID GROUP BY Depa
rtment.DeptName;
+----------+--------------+
| DeptName | HighestMarks |
+----------+--------------+
| CSE      |           85 |
| ECE      |           95 |
| EEE      |           88 |
+----------+--------------+
3 rows in set (0.00 sec)

mysql> SELECT StudentName, Marks FROM Student WHERE Marks > ( SELECT AVG(Marks) FROM Student );
+-------------+-------+
| StudentName | Marks |
+-------------+-------+
| Priya       |    90 |
| Sneha       |    88 |
| Kiran       |    95 |
+-------------+-------+
3 rows in set (0.04 sec)

mysql> SELECT AVG(Marks) FROM Student;
+------------+
| AVG(Marks) |
+------------+
|    86.3333 |
+------------+
1 row in set (0.00 sec)

mysql> SELECT StudentName, Marks FROM Student WHERE Marks > 86.33;
+-------------+-------+
| StudentName | Marks |
+-------------+-------+
| Priya       |    90 |
```

```
mysql> SELECT StudentName, Marks FROM Student WHERE Marks > 86.33;
+-------------+-------+
| StudentName | Marks |
+-------------+-------+
| Priya       |    90 |
| Sneha       |    88 |
| Kiran       |    95 |
+-------------+-------+
3 rows in set (0.00 sec)

mysql> SELECT Department.DeptName, AVG(Student.Marks) AS AverageMarks FROM Student INNER JOIN Department ON Student.DeptID = Department.DeptID GROUP BY Depa
rtment.DeptName HAVING AVG(Student.Marks) > 85;
+----------+--------------+
| DeptName | AverageMarks |
+----------+--------------+
| ECE      |      92.5000 |
| EEE      |      88.0000 |
+----------+--------------+
2 rows in set (0.01 sec)
```

10. **Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning.**
    Bloom's Level: BL5 – Evaluate

ANS:

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database department;
Query OK, 1 row affected (0.06 sec)

mysql> use department;
Database changed
mysql> CREATE TABLE Department ( DeptID INT PRIMARY KEY, DeptName VARCHAR(30) );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Student ( StudentID INT PRIMARY KEY, StudentName VARCHAR(30), DeptID INT, Marks INT, FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
Query OK, 0 rows affected (0.12 sec)

mysql> INSERT INTO Department VALUES (10,'CSE'), (20,'ECE'), (30,'EEE');
Query OK, 3 rows affected (0.09 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Student VALUES (101,'Rahul',10,85), (102,'Priya',20,90), (103,'Arun',10,78), (104,'Sneha',30,88);
Query OK, 4 rows affected (0.03 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT StudentName, DeptName FROM Student INNER JOIN Department ON Student.DeptID=Department.DeptID;
+-------------+----------+
| StudentName | DeptName |
+-------------+----------+
| Rahul       | CSE      |
| Arun        | CSE      |
| Priya       | ECE      |
| Sneha       | EEE      |
+-------------+----------+
4 rows in set (0.00 sec)

mysql> SELECT StudentName FROM Student WHERE DeptID= ( SELECT DeptID FROM Department WHERE DeptName='CSE' );
+-------------+
| StudentName |
+-------------+
| Rahul       |
| Arun        |
+-------------+
2 rows in set (0.08 sec)

mysql> CREATE VIEW Student_Department_View AS SELECT Student.StudentName, Department.DeptName FROM Student INNER JOIN Department ON Student.DeptID=Departmen
t.DeptID;
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> CREATE VIEW Student_Department_View AS SELECT Student.StudentName, Department.DeptName FROM Student INNER JOIN Department ON Student.DeptID=Departmen
t.DeptID;
Query OK, 0 rows affected (0.02 sec)

mysql> SELECT * FROM Student_Department_View;
+-------------+----------+
| StudentName | DeptName |
+-------------+----------+
| Rahul       | CSE      |
| Arun        | CSE      |
| Priya       | ECE      |
| Sneha       | EEE      |
+-------------+----------+
4 rows in set (0.05 sec)
```

Justification

- The query is executed frequently.
- View avoids rewriting the same SQL statement.
- Simplifies query execution.
- Improves security by exposing only required columns.